

Java Concurrency & Multithreading

Complete Course: Zero to Hero

Overview

These detailed notes cover the core concepts of Java Concurrency and Multithreading, focusing on real-world engineering practices, core APIs, and common concurrency patterns. They provide a comprehensive understanding of multithreading from first principles to advanced abstractions in the `java.util.concurrent` package.

1. Introduction: Process vs. Thread

Process	Thread
An independent program in execution.	A lightweight sub-process or path of execution within a process.
Has its own isolated memory space.	Shares the memory space (heap) of the parent process.
Heavyweight; context switching is expensive.	Lightweight; context switching is much faster.

2. Creating Threads in Java

There are two primary ways to create a thread in Java:

Method 1: Extending the Thread Class

```
class MyThread extends Thread {
    @Override
    public void run() {
        System.out.println("Thread is running...");
    }
}
// Usage:
MyThread t1 = new MyThread();
t1.start(); // Always call start(), not run()!
```

Method 2: Implementing the Runnable Interface (Preferred)

This is the recommended approach because Java does not support multiple inheritance. By implementing `Runnable`, your class can still extend another class.

```
class MyRunnable implements Runnable {
    @Override
    public void run() {
        System.out.println("Runnable thread is running...");
    }
}
```

```

    }
}
// Usage:
Thread t2 = new Thread(new MyRunnable());
t2.start();

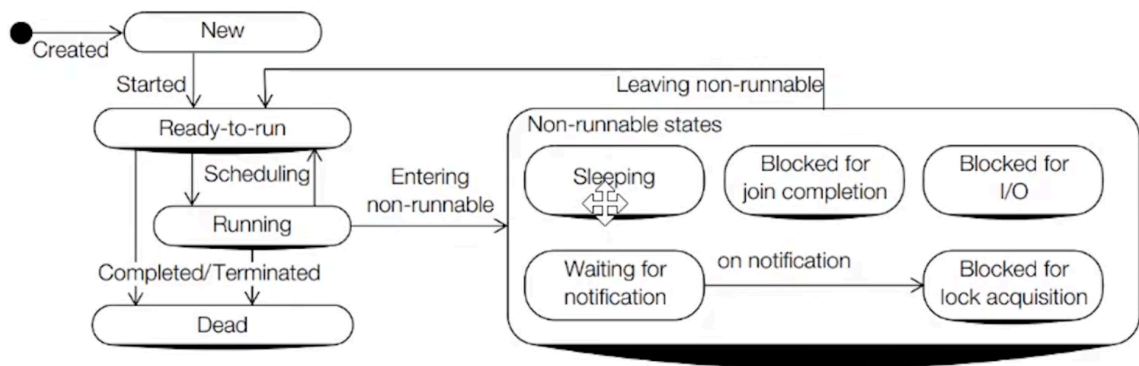
```

Note: Calling `run()` directly instead of `start()` does not create a new thread. It simply executes the `run` method synchronously in the current thread.

3. Thread Lifecycle and States

Thread Transitions

Thread States



A thread in Java goes through several states during its lifecycle, governed by the JVM and OS thread scheduler:

- **NEW:** The thread is created but `start()` has not been called yet.
- **RUNNABLE:** The thread is ready to run and waiting for CPU time, or is actively executing.
- **BLOCKED:** The thread is waiting to acquire a monitor lock to enter a synchronized block/method.
- **WAITING:** The thread is waiting indefinitely for another thread to perform a specific action (e.g., via `Object.wait()` or `Thread.join()`).
- **TIMED_WAITING:** Similar to waiting, but for a specified waiting time (e.g., `Thread.sleep(ms)` or `Object.wait(ms)`).
- **TERMINATED:** The thread has finished executing its `run()` method.

4. Concurrency Issues & Synchronization

Race Conditions

Occurs when multiple threads share data and attempt to modify it concurrently, leading to

unpredictable or incorrect results. To prevent this, we use synchronization to enforce mutual exclusion.

The synchronized Keyword

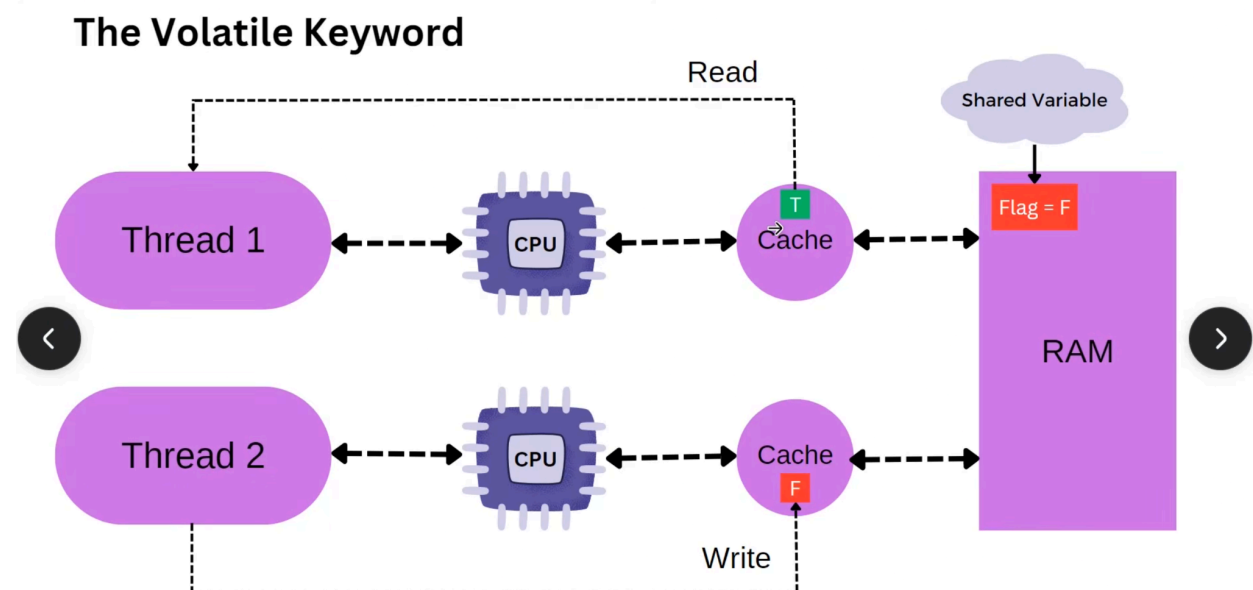
- **Synchronized Method:** Locks the entire object instance. Only one thread can execute any synchronized method of that object at a time.
- **Synchronized Block:** Preferred over method synchronization because it limits the scope of the lock, improving performance.

```
public void increment() {  
    synchronized(this) {  
        count++;  
    }  
}
```

Object Level vs. Class Level Locks

- **Object Level Lock:** Acquired on the instance of the class (this). Multiple instances have independent locks.
- **Class Level Lock:** Acquired on the Class object (e.g., MyClass.class). Used primarily for static synchronized methods or blocks to lock across all instances.

5. The Volatile Keyword

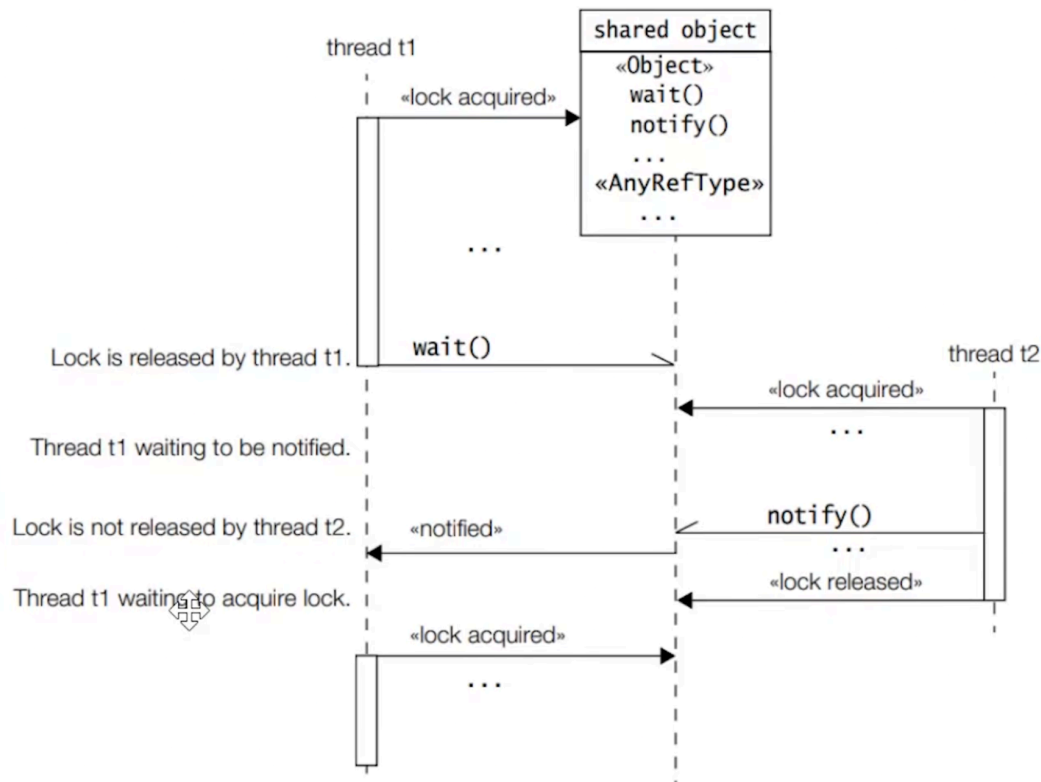


In a multicore processor, threads can cache variables in their local CPU cache. If one thread updates a variable, another thread might still see the old cached value. This is the **visibility problem**.

The volatile keyword guarantees that reads and writes to a variable go directly to the main

memory, bypassing the CPU cache. However, it does **not** guarantee atomicity (e.g., `count++` is still unsafe even if `count` is volatile).

6. Inter-Thread Communication



Threads often need to coordinate. This is done using methods from the `Object` class, which must be called from within a synchronized context.

- `wait()`: Causes the current thread to release the lock and wait until another thread wakes it up.
- `notify()`: Wakes up a single thread that is waiting on this object's monitor.
- `notifyAll()`: Wakes up all threads waiting on this object's monitor.

Producer-Consumer Problem

A classic concurrency scenario where a `Producer` thread creates data and puts it in a shared buffer, while a `Consumer` thread removes it. `wait()` and `notify()` are heavily used here to ensure the producer pauses when the buffer is full and the consumer pauses when the buffer is empty.

7. Essential Thread Methods

- `Thread.sleep(long ms)`: Pauses execution for a specified time. It does **not** release locks.
- `Thread.yield()`: A hint to the scheduler that the current thread is willing to yield its current use of a processor.

- `thread.join()`: Forces the current thread to wait until the thread it joined finishes execution.

8. High-Level Concurrency: The Executor Framework

Manually creating threads is expensive and hard to manage. The `java.util.concurrent` package introduces Thread Pools to manage worker threads efficiently.

Thread Pools

A pool of pre-created threads that execute tasks sequentially from a queue. Advantages include resource management, better performance, and avoiding thread exhaustion.

- `Executors.newFixedThreadPool(n)`: A pool with a fixed number of threads.
- `Executors.newCachedThreadPool()`: A pool that creates new threads as needed but reuses previously constructed threads when available.
- `Executors.newSingleThreadExecutor()`: A pool with exactly one thread.

Runnable vs. Callable

Runnable	Callable
Has a <code>run()</code> method.	Has a <code>call()</code> method.
Cannot return a result (returns void).	Can return a generic result (V).
Cannot throw checked exceptions.	Can throw checked exceptions.

Futures

When you submit a Callable to an Executor Service, it returns a Future object representing the pending result of the task.

```
ExecutorService executor = Executors.newFixedThreadPool(2);
Future<Integer> future = executor.submit(new MyCallable());
// ... do other things ...
Integer result = future.get(); // This is a blocking call!
executor.shutdown();
```

9. Deadlocks

A Deadlock occurs when two or more threads are blocked forever, each waiting for the other to release a lock.

Conditions for Deadlock (Hold and Wait): Thread A holds Lock 1 and waits for Lock 2. Thread B holds Lock 2 and waits for Lock 1.

Prevention Strategy: The most common way to prevent deadlocks is *Lock Ordering* — ensuring that all threads acquire multiple locks in the exact same order.

10. Advanced Locks

Beyond the `synchronized` keyword, Java provides explicit locks via the `java.util.concurrent.locks` package.

- **ReentrantLock:** Offers features like fairness policies (giving the lock to the longest-waiting thread), interruptible lock waits, and the ability to try acquiring a lock without blocking indefinitely.
- **ReadWriteLock:** Maintains a pair of associated locks. The read lock may be held simultaneously by multiple reader threads, while the write lock is exclusive. Ideal for read-heavy operations.